
Classification Training & Evaluation Lab (Practical)

Welcome to the practical lab for Classification Training & Evaluation! In this session, you will write a standalone Python script to train a **Logistic Regression** and **Decision Tree** classification model using **Scikit-Learn**, generate a **Confusion Matrix**, and analyze performance using accuracy, precision, recall, and F1-score.

1. Concept Overview

In this lab, you will:

- Prepare a binary classification dataset (e.g., predicting whether a customer will churn or stay).
 - Train a Scikit-Learn `LogisticRegression` model on training features.
 - Generate and interpret a **Confusion Matrix** to inspect True Positives, True Negatives, False Positives, and False Negatives.
 - Print a complete classification report detailing precision, recall, and F1-scores.
-

2. Step-by-Step Lab Instructions

Step 1: Create the Classification Script (`train_classification.py`)

Create a new file named `train_classification.py` in your project folder and add the following code:

```
# train_classification.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score

# 1. Create a sample classification dataset (Monthly Charges & Tenure vs
Churn)
```

```

# Target: churn (0 = Retained, 1 = Churned)
data = {
    "monthly_charges": [30.5, 85.0, 92.5, 20.0, 95.0, 45.0, 88.0, 22.5,
90.0, 78.0],
    "tenure_months": [24, 3, 2, 36, 1, 18, 4, 30, 2, 5],
    "churn": [0, 1, 1, 0, 1, 0, 1, 0, 1, 1]
}

df = pd.DataFrame(data)

# Separate features (X) and target labels (y)
X = df[["monthly_charges", "tenure_months"]]
y = df["churn"]

# 2. Split dataset into Training (70%) and Testing (30%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# 3. Initialize and train the Logistic Regression classifier
model = LogisticRegression()
model.fit(X_train, y_train)

# 4. Make predictions on the test set
y_pred = model.predict(X_test)

# 5. Generate Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("=== Confusion Matrix ===")
print(conf_matrix)
print("\n[ Format: ]")
print("[ True Negatives (TN)  False Positives (FP) ]")
print("[ False Negatives (FN)  True Positives (TP) ]\n")

# 6. Generate Detailed Classification Report
print("=== Classification Report ===")
print(classification_report(y_test, y_pred, target_names=["Retained (0)",
"Churned (1)"]))

# 7. Calculate Accuracy
acc = accuracy_score(y_test, y_pred)
print(f"Overall Accuracy: {acc * 100:.2f}%")

```

Step 2: Run Your Script

Open your terminal, ensure your virtual environment is active, and execute the script:

```
python train_classification.py
```

3. Analyzing the Confusion Matrix

When you run the script, the confusion matrix outputs a 2×2 grid comparing the actual test labels against your model's predictions:

- **True Negatives (Top-Left):** Cases where the model correctly predicted customers who stayed (0).
- **False Positives (Top-Right):** Cases where the model incorrectly predicted a customer would churn when they actually stayed (Type I error).
- **False Negatives (Bottom-Left):** Cases where the model missed a churned customer, predicting they stayed when they actually left (Type II error—often the most costly mistake in business scenarios).
- **True Positives (Bottom-Right):** Cases where the model correctly identified customers who churned (1).

The generated classification report will give you the precise **Precision**, **Recall**, and **F1-Score** for both classes, allowing you to evaluate how well your model performs before integrating it into FastAPI.

Decision Tree Classification

1. Concept Overview

In this lab, you will:

- Build a binary classification script using Scikit-Learn's `DecisionTreeClassifier`.
 - Train the model on feature splits (e.g., customer tenure and monthly charges).
 - Evaluate tree-based predictions using a confusion matrix and classification report.
-

2. Step-by-Step Lab Instructions

Step 1: Create the Decision Tree Script (`train_decision_tree.py`)

Create a new file named `train_decision_tree.py` in your project folder and add the following code:

```
# train_decision_tree.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score

# 1. Sample classification dataset (Monthly Charges & Tenure vs Churn)
```

```

data = {
    "monthly_charges": [30.5, 85.0, 92.5, 20.0, 95.0, 45.0, 88.0, 22.5,
90.0, 78.0],
    "tenure_months": [24, 3, 2, 36, 1, 18, 4, 30, 2, 5],
    "churn": [0, 1, 1, 0, 1, 0, 1, 0, 1, 1]
}

df = pd.DataFrame(data)

# Separate features (X) and target labels (y)
X = df[["monthly_charges", "tenure_months"]]
y = df["churn"]

# 2. Split dataset into Training (70%) and Testing (30%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# 3. Initialize and train the Decision Tree Classifier
# max_depth limits how deep the tree can grow to prevent overfitting
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)

# 4. Make predictions on the test set
y_pred = model.predict(X_test)

# 5. Generate Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("=== Confusion Matrix (Decision Tree) ===")
print(conf_matrix)
print("\n[ Format: ]")
print("[ True Negatives (TN)  False Positives (FP) ]")
print("[ False Negatives (FN)  True Positives (TP) ]\n")

# 6. Generate Detailed Classification Report
print("=== Classification Report ===")
print(classification_report(y_test, y_pred, target_names=["Retained (0)",
"Churned (1)"]))

# 7. Calculate Accuracy
acc = accuracy_score(y_test, y_pred)
print(f"Overall Accuracy: {acc * 100:.2f}%")

```

Step 2: Run Your Script

Open your terminal, ensure your virtual environment is active, and execute the script:

```
python train_decision_tree.py
```

3. Analyzing the Output

When you execute the script:

- **Decision Rules:** Unlike logistic regression which fits a single linear boundary, the decision tree recursively splits data using threshold conditions on `monthly_charges` and `tenure_months`.
 - **Confusion Matrix & Metrics:** Review how the precision and recall adjust compared to the linear model. Decision trees can model complex non-linear patterns, but controlling `max_depth` is crucial to prevent memorizing the training set (*overfitting*).
-

4. Reference Links & Further Reading

- **Scikit-Learn Decision Trees Guide:** Official documentation on classification and regression trees.
 - **Scikit-Learn Metrics API Reference:** Comprehensive overview of evaluation metrics and scoring functions.
 - - **Scikit-Learn Confusion Matrix Documentation:** API guide for calculating classification matrices.
 - **Scikit-Learn Classification Report Guide:** Learn how precision, recall, and F1-scores are computed automatically in Python.
-